# Software Quality Assurance Plan and Report

| | |
|---|---|
| Projekttitel | Feels Like |
| Erstellt am | 11. Februar 2026 |
| Zuletzt aktualisiert von | Julian Stengele |
| Zuletzt aktualisiert um | 5. März 2026 10:12 |
| Überprüfer/-innen | Julian Stengele Atu Isabella Schwarz Sophie Lazarjan |
| Erstellt von | Isabella Schwarz |
| Kategorie | Strategiedokument |

## Versionsverwaltung

| Version | Datum | Änderungen |
|---|---|---|
| V 1.0.0 | 11. Februar 2026 | Initialversion |
| V 1.1.0 | 05. März 2026 | Erste Inhalte im Report-Teil |

# Plan

▼ 1. Einleitung und Anwendungsbereich

### 1.1 Projektbeschreibung

Dieser Software Quality Assurance Plan (SQAP) definiert die Qualitätssicherungsmaßnahmen für das Projekt "Feels Like", eine Webanwendung zur Verwaltung und Analyse von sportlichen

Trainingsdaten. Die Architektur basiert auf einem React-Frontend, einem Node.js/Express-Backend und einer SQLite-Datenbank.

### 1.2 Zielsetzung und Geltungsbereich

Das Ziel dieses Dokuments ist es, einen systematischen Ansatz zur Sicherstellung einer hohen Softwarequalität über den gesamten Entwicklungszyklus hinweg zu etablieren. Der Plan soll gewährleisten, dass die implementierten Features (Sprints) fehlerfrei, sicher und wartbar sind. Der Geltungsbereich dieses Plans umfasst den gesamten Quellcode (Frontend & Backend), die Datenbankstruktur sowie die begleitende Softwaredokumentation.

▼ 2. Normative Referenzen

Die Qualitätssicherung in diesem Projekt orientiert sich an etablierten Industrie-Standards der Webentwicklung sowie intern festgelegten Richtlinien:

- **Externe Best Practices:** Orientierung an den Prinzipien der agilen Softwareentwicklung und den "Clean Code"-Standards (z. B. saubere Trennung von Logik und Darstellung, aussagekräftige Benennungen).
- **Interne Richtlinien:**
  - **Code Conventions:** Strikte Einhaltung der definierten `ESLint` Regeln im gesamten Monorepo zur automatischen Fehlervermeidung.
  - **Versionskontrolle:** Nutzung eines agilen Git-Workflows (Feature-Branches) über GitHub.
- **Mitgeltende Projektdokumente:**
  - Software Detailed Design Document (SDD)
  - Software Verification Plan (SVP)
  - Pflichtenheft / Anforderungsanalyse (Use Cases)

▼ 3. Audit- und Bewertungskriterien

Um die Qualität objektiv zu messen und zu überwachen, werden folgende Audits und Metriken definiert:

### 3.1 Geplante Audits (Prüfungen)

- **Laufendes Code-Audit (Peer Reviews):** Vor jedem Merge eines Feature-Branches in den Hauptentwicklungszweig (`dev`) muss ein Code-Review durch ein anderes Teammitglied durchgeführt werden (**Vier-Augen-Prinzip**). Fokus liegt auf Sicherheitsaspekten (z. B. keine Plaintext-Passwörter) und Architekturvorgaben (z. B. MVC-Trennung).
- **Meilenstein-Audit (Sprint Review):** Am Ende jedes Sprints wird die Software gegen die definierten Use Cases (Akzeptanzkriterien) aus dem Pflichtenheft abgeglichen. Hierbei werden die im SVP definierten Systemtests (z. B. TC-01 Login Success) manuell abgenommen.
- **Agiles Projektmanagement mit GitHub Projects:**
  Das Entwicklungsteam nutzt das GitHub Project Board als Scrum-Board zur Sprint-Planung und Fortschrittsverfolgung. Jede Aufgabe wird als GitHub Issue angelegt und durchläuft den Workflow Backlog → Ready → In Progress → In Review → Done. Issues sind Sprint-Iterationen zugeordnet und werden mit Feature-Branches verknüpft (Konvention: feature/<beschreibung>). Die Abarbeitung erfolgt nach dem Pull-Request-Verfahren mit Code Review durch ein weiteres Teammitglied, bevor ein Branch in dev oder main gemerged wird. Dies stellt sicher, dass Änderungen mindestens dem Vier-Augen-Prinzip entsprechen.

▼ 4. Dokumentation und Berichterstattung

Qualitätssicherung muss nachvollziehbar sein. Folgende Regeln gelten für die Dokumentation und Freigabe:

## 4.1 Reviews und Fehler-Tracking

- **GitHub Issues:** Alle identifizierten Fehler (Bugs) oder Architektur-Verstöße werden zentral als Issue in GitHub erfasst und mit dem Label `bug` versehen.
- **Pull Request (PR) Dokumentation:** Jede Code-Freigabe muss über einen PR in GitHub erfolgen. Der Reviewer dokumentiert seine Zustimmung explizit durch ein "Approve" (Freigabe) in der GitHub-Oberfläche.

## 4.2 Freigabebedingungen (Release Readiness)

Damit ein neuer Meilenstein (z. B. "Sprint 1 abgeschlossen") formal freigegeben werden kann, müssen folgende Kriterien erfüllt sein:

1. Der lokale Build-Prozess (`npm run build` im Client) läuft fehlerfrei durch.

2. Die im SVP definierten Systemtests für die entsprechenden Use Cases wurden erfolgreich durchgeführt.
3. Die Umgebungsvariablen (`.env`) und sensiblen Daten sind korrekt abgesichert und nicht im Repository versioniert.

### 4.3 Berichterstattung (SQAR)

Am Ende des Projekts wird auf Basis dieses Plans ein **Software Quality Assurance Report (SQAR)** erstellt. Dieser Report dokumentiert die Ergebnisse der Audits, wertet die Metriken aus und bestätigt die Einhaltung der hier definierten Qualitätsstandards.

# Report

▼ 1. Zusammenfassung der Qualitätssicherungstätigkeiten

**Unit Tests:** Für die Service-Schicht des Backends (`trainingService.js`) wurden 30 automatisierte Unit Tests mit Jest implementiert. Diese decken die vollständige CRUD-Logik sowie die Eingabevalidierung ab.

**Statische Codeanalyse:** ESLint ist sowohl im Client- als auch im Server-Projekt konfiguriert und wird zur automatischen Erkennung von Code-Stil-Verstößen und potenziellen Fehlern eingesetzt.

**Code Reviews:** Alle Änderungen wurden über Feature-Branches entwickelt und vor dem Merge in den Hauptbranch durch Pull Requests mit Review freigegeben (Vier-Augen-Prinzip gemäß SQAP Abschnitt 3.1).

▼ 2. Konformität mit dem SQAP

Die im SQAP definierten Maßnahmen wurden wie folgt eingehalten:

- **Peer Reviews (Abschnitt 3.1):** Feature-Branches wurden über GitHub Pull Requests gemerged. Code Reviews wurden vor jedem Merge durch ein Teammitglied durchgeführt.
- **Fehler-Tracking (Abschnitt 4.1):** Identifizierte Bugs wurden als GitHub Issues mit dem Label `bug` erfasst und nachverfolgt.
- **Freigabebedingungen (Abschnitt 4.2):** Der Client-Build (`npm run build`) läuft fehlerfrei. Sensible Daten (JWT_SECRET) werden über Umgebungsvariablen verwaltet und sind nicht im Repository versioniert.

- **Meilenstein-Audits (Abschnitt 3.1):** Am Ende jedes Sprints wurde die Software gegen die definierten Use Cases aus dem Pflichtenheft abgeglichen.

▼ 3. Ergebnisse der Verifikations- und Validierungsmaßnahmen

**Unit Tests (trainingService.test.js) — 30 Testfälle:**

| Bereich | Anzahl Tests | Ergebnis |
|---|---|---|
| Validierung – Pflichtfelder (datum, sportart, dauer, feelsLikeScore) | 5 | Bestanden |
| Validierung – Grenzwerte feelsLikeScore (0, 1, 10, 11) | 4 | Bestanden |
| Validierung – Grenzwerte dauer (0, 1, negativ) | 3 | Bestanden |
| Validierung – Erlaubte Sportarten | 2 | Bestanden |
| Validierung – Distanz (negativ, null, positiv) | 3 | Bestanden |
| Training erfassen (Erfolg + Fehler) | 3 | Bestanden |
| Alle Trainings laden (mit/ohne Daten) | 2 | Bestanden |
| Training ändern (Erfolg, nicht gefunden, unberechtigt, Validierung) | 4 | Bestanden |
| Training löschen (Erfolg, nicht gefunden, unberechtigt) | 3 | Bestanden |
| Edge Case: leere Notiz erlaubt | 1 | Bestanden |

**Systemtests:** Die im SVP definierten manuellen Testfälle (TC-SYS-01 bis TC-SYS-04) für Login, Training erfassen, Training bearbeiten und Training löschen wurden manuell verifiziert und bestanden.

▼ 4. Audit- und Review-Ergebnisse

**Code Reviews:**

Pull Requests auf den einzelnen Feature-Branches wurden durch Teammitglieder reviewed. Dabei wurden keine kritischen Sicherheitsmängel oder Architekturverstöße festgestellt.

**Feststellungen aus Reviews:**

- Passwörter werden korrekt mit bcrypt gehasht (kein Plaintext) — konform mit SQAP.
- JWT-Tokens werden als HttpOnly-Cookies gesetzt — konform mit Sicherheitsanforderungen.

- Die MVC-Trennung (Controller → Service → Model) wird konsequent eingehalten.

▼ 5. Qualitätsmetriken und Leistungsbewertung

| Metrik | Wert | Bewertung |
|---|---|---|
| Anzahl Unit Tests | 30 | Service-Schicht vollständig abgedeckt |
| Unit Test Erfolgsrate | 100 % (30/30) | Ziel erreicht |
| ESLint-Verstöße (Server) | 0 | Ziel erreicht |
| ESLint-Verstöße (Client) | 0 | Ziel erreicht |
| Offene Bugs | 0 | Ziel erreicht |

▼ 6. Lessons Learned und Empfehlungen

**Positiv:**

- Die frühe Einführung von ESLint hat Code-Stil-Konflikte im Team vermieden.
- Die Auslagerung der Geschäftslogik in eine Service-Schicht hat die Testbarkeit deutlich verbessert (Unit Tests ohne HTTP-Server möglich).
- Der Feature-Branch-Workflow hat paralleles Arbeiten ohne Konflikte ermöglicht.

**Empfehlungen für zukünftige Projekte:**

- Automatisierte Tests von Beginn an für alle Schichten (inkl. Frontend) einplanen.
- Code-Coverage-Metriken als festes Qualitätsziel definieren.
- Sprint Reviews formal protokollieren, um die Nachvollziehbarkeit zu verbessern.

▼ 7. Schlussfolgerungen und Freigabeempfehlungen

Die im SQAP definierten Qualitätsmaßnahmen wurden eingehalten. Die implementierte Software ist durch Unit Tests und manuelle Systemtests verifiziert. Sicherheitsrelevante Anforderungen (Passwort-Hashing mit bcrypt, JWT als HttpOnly-Cookie, Umgebungsvariablen für sensible Daten) sind korrekt umgesetzt.

**Freigabeempfehlung:** Die Software erfüllt die definierten Freigabebedingungen (Abschnitt 4.2 des SQAP) und wird zur Abnahme

freigegeben.